Guia completa de FacciStats API

Explicacion tecnica, paso a paso, para ejecutar, entender, probar y defender el proyecto.

Proyecto local:
C:\Users\caleb\OneDrive\Escritorio\TecWeb\pro3\TrabajoECU3\Almacenamiento-y-proteccion-de-datos-jpah

Repositorio remoto:
https://github.com/pabloayavirih-bot/Almacenamiento-y-protecci-n-de-datos-jpah.git

Esta guia explica la version organizada por capas. No contiene contrasena de MySQL, cadena de conexion real ni clave AES: esos valores estan solo en Secret Manager local.

Indice y proposito

- Que resuelve el proyecto.
- Como abrir, configurar y ejecutar.
- Mapa de carpetas y archivos.
- Flujo interno de una peticion.
- Modelo de datos y migraciones.
- Proteccion de datos.
- Todos los endpoints.
- Estadisticas propias.
- Pruebas, defensa y solucion de problemas.

Como usar la guia: cada seccion explica que hace una parte, por que existe y donde esta. Las rutas indicadas son relativas a la carpeta raiz del proyecto.

1. Que resuelve FacciStats

FacciStats es una API REST para administrar personajes, cartas, eventos y usuarios de un catalogo galactico. Su funcionalidad propia transforma esos registros en estadisticas por faccion: personajes, vivos, muertos, cartas, poder promedio, eventos ganados y porcentaje de bajas.

Una API no es una pagina web comun: recibe solicitudes HTTP, consulta o modifica MySQL y devuelve JSON. Swagger es la interfaz que documenta y permite probar cada operacion desde el navegador.

Parte | Que aporta | Ubicacion

CRUD | Crear, leer, actualizar y eliminar personajes, cartas, eventos y usuarios. | Controllers/PersonajesController.cs, CartasController.cs, EventosController.cs, UsuariosController.cs.

Componente propio | Filtros, paginacion, ranking, comparacion y estadisticas por faccion. | Controllers/FaccionesController.cs y Services/ApiService.cs, metodo Estadisticas().

Persistencia | Entidades almacenadas como tablas relacionales en MySQL. | Data/AppDbContext.cs y Migrations/.

Proteccion | Secretos fuera de Git, hash de contrasena y cifrado de datos sensibles. | Program.cs, Services/CifradoService.cs, controladores de personajes y usuarios.

Conceptos esenciales

- REST: una URL representa un recurso y el metodo HTTP expresa la accion: GET lee, POST crea, PUT actualiza y DELETE elimina.
- Endpoint: combinacion de metodo y ruta, por ejemplo GET /api/v1/personajes.
- JSON: formato estructurado de respuesta, por ejemplo {"id":1,"nombre":"Astra"}.
- Entidad: clase que Entity Framework Core convierte en tabla.
- DTO: objeto de entrada/salida que limita exactamente los datos que la API acepta o expone.
- DI: inyeccion de dependencias: ASP.NET entrega un servicio al controlador sin que este lo construya manualmente.

2. Como abrir, configurar y ejecutar

Requisitos

- .NET SDK 10 instalado: verifica con dotnet --version.
- MySQL Server 8.0 activo en el puerto 3306. Este proyecto usa MySQL, no MariaDB ni XAMPP.
- Base facci_stats_db y usuario de aplicacion creados localmente.
- Paquetes restaurados con dotnet restore la primera vez.

Ubicacion y secretos

```
cd C:\Users\caleb\OneDrive\Escritorio\TecWeb\pro3\TrabajoECU3\Almacenamiento-y-proteccion-de-datos-jpah
```

En Git Bash se puede usar cd ~/OneDrive/Escritorio/TecWeb/pro3/TrabajoECU3/Almacenamiento-y-proteccion-de-datos-jpah. El identificador de Secret Manager esta en FacciStats.csproj, propiedad UserSecretsId. Desde la carpeta del proyecto se configuran valores privados:

```
dotnet user-secrets init
dotnet user-secrets set "ConnectionStrings:DefaultConnection"
"Server=localhost;Port=3306;Database=facci_stats_db;User=facci_user;Password=TU_CLAVE"
dotnet user-secrets set "Cifrado:Clave" "CLAVE_BASE64_DE_32_BYTES"
```

Para generar una clave AES valida en PowerShell: $bytes = New-Object byte[] 32; [Security.Cryptography.RandomNumberGenerator]::Create().GetBytes($bytes); [Convert]::ToBase64String($bytes).

No guardes valores reales en appsettings.json, capturas o Git. Ese archivo solo conserva Logging y AllowedHosts. .gitignore protege artefactos locales como bin/ y obj/.

Migrar e iniciar

```
dotnet ef database update
```

El comando recorre Migrations/, actualiza MySQL y registra lo aplicado en la tabla __EFMigrationsHistory.

En PowerShell:

```
$env:ASPNETCORE_ENVIRONMENT="Development"
dotnet run
```

En Git Bash:

```
ASPNETCORE_ENVIRONMENT=Development dotnet run
```

La sintaxis $env: es exclusiva de PowerShell; por eso falla en Git Bash. Abre luego http://localhost:5000/swagger/index.html, o el puerto que muestre la consola. Deten la API con Ctrl + C; despues de editar codigo debes iniciar otra vez.

3. Mapa de carpetas y archivos

```
Almacenamiento-y-proteccion-de-datos-jpah/
|- Controllers/ Endpoints HTTP, uno por area
|- Data/ AppDbContext y reglas de EF Core
|- Dtos/ Requests y respuestas de la API
|- Migrations/ Historial versionado del esquema MySQL
|- Models/ Entidades convertidas en tablas
|- Properties/ Perfil de lanzamiento local
|- Services/ Cifrado y logica reutilizable
|- docs/ Fuente HTML de esta guia
|- output/pdf/ PDF final de esta guia
|- Program.cs Punto de arranque
|- appsettings.json Configuracion no secreta
|- FacciStats.csproj Paquetes, framework y User Secrets
|- FacciStats.http Solicitudes de ejemplo
|- README.md Instrucciones rapidas
`- USO_DE_IA.md Declaracion de uso puntual de IA
```

Archivo | Responsabilidad exacta

Program.cs | Construye la app, carga conexion, registra EF Core, controladores, Swagger, PasswordHasher, ApiService y CifradoService; al final mapea controladores y ejecuta.

Models/Entities.cs | Declara Personaje, Carta, Evento y Usuario, sus campos y navegaciones.

Data/AppDbContext.cs | Expone las tablas con DbSet y define longitudes, obligatorios, indice unico y relacion 1:N.

Dtos/Requests.cs | Define entradas de POST/PUT y las anotaciones de validacion.

Dtos/Responses.cs | Define PersonajeDto y FaccionDto, contratos de salida.

Services/CifradoService.cs | Cifra y descifra texto con AES-GCM y una clave obtenida por configuracion.

Services/ApiService.cs | Centraliza el mapeo/descifrado, estadisticas y creacion segura de usuario.

Controllers/*.cs | Reciben HTTP, aplican reglas, consultan EF Core/servicios y eligen el estado HTTP.

En FacciStats.csproj, TargetFramework define .NET 10; Nullable ayuda a detectar nulos; Pomelo es el proveedor que traduce EF Core a SQL para MySQL. Los otros paquetes habilitan herramientas EF, OpenAPI y Swagger UI.

4. Que ocurre en una peticion

Cliente o Swagger -> Controller -> Servicio cuando corresponde -> AppDbContext / EF Core -> MySQL -> JSON y codigo HTTP

- Swagger, un navegador o cliente HTTP envia una URL.
- ASP.NET encuentra el controlador por [Route(...)] y el metodo por [HttpGet], [HttpPost], etc.
- ASP.NET convierte el JSON a un Request DTO. [ApiController] hace que anotaciones como [Required] generen 400 cuando la entrada no es valida.
- El controlador ejecuta reglas de negocio: una carta, por ejemplo, exige un personaje existente.
- EF Core convierte operaciones LINQ en SQL y AppDbContext las ejecuta en MySQL.
- Si hay texto sensible, CifradoService cifra antes de guardar o descifra antes de responder.
- El resultado se serializa a JSON con un codigo HTTP adecuado.

Ejemplo completo: POST de personaje

Se implementa en Controllers/PersonajesController.cs, metodo Crear. Recibe PersonajeRequest, aplica Trim() a nombre/faccion, cifra la biografia si existe, ejecuta db.Add y SaveChangesAsync. Luego responde 201 mediante CreatedAtAction, que ademas aporta la URL para consultar al nuevo recurso. Antes de responder llama service.Map, por lo que la biografia vuelve descifrada al cliente.

AsNoTracking() aparece en muchas lecturas. Significa que EF Core no vigilara cambios de esos objetos: es apropiado y mas liviano cuando solo se van a mostrar. Para editar se usa FindAsync, que entrega una entidad seguida por el contexto; SaveChangesAsync guarda las modificaciones.

5. Modelo de datos y migraciones

Entidad / tabla | Campos principales | Sentido

Personaje / Personajes | Id, Nombre, Faccion, EstaMuerto, BiografiaSecreta | Personaje del catalogo. Biografia se guarda cifrada; el booleano permite contar vivos y muertos.

Carta / Cartas | Id, PersonajeId, Nombre, Poder | Carta de un personaje; su poder alimenta las estadisticas.

Evento / Eventos | Id, Nombre, Fecha, FaccionGanadora | Resultado para contar victorias por faccion.

Usuario / Usuarios | Id, Nombre, Email, PasswordHash, Telefono | Demuestra hash de password y cifrado de dato personal.

Las cuatro clases estan en Models/Entities.cs. "" evita nulos en texto requerido. El signo ? significa que algo puede ser nulo, como Telefono o BiografiaSecreta.

Relacion uno a muchos

Un Personaje (1) <--- tiene muchas ---> Cartas (N)

En Personaje, List<Carta> Cartas es la navegacion hacia sus cartas. En Carta, PersonajeId es llave foranea y Personaje navega al padre. En Data/AppDbContext.cs, dentro del bloque de Carta, HasOne(...).WithMany(...).HasForeignKey(...).OnDelete(DeleteBehavior.Restrict) establece la relacion. Restrict impide borrar un personaje que tiene cartas; el DELETE tambien lo detecta y responde 400.

Reglas de AppDbContext

El metodo OnModelCreating de Data/AppDbContext.cs fija maximos de longitud, obligatoriedad, email unico con HasIndex(x => x.Email).IsUnique(), password hash requerido y columnas de datos cifrados. Sus propiedades DbSet representan las tablas que EF Core administra.

Migraciones

Archivo | Que hace | Importancia

Migrations/20261109000100_Inicial.cs | Crea Personajes, Cartas, Eventos, Usuarios, indices y llave foranea. | Primera version reproducible del esquema.

Migrations/20261109000200_AgregarBiografiaCifrada.cs | Agrega BiografiaSecreta. | Prueba que el modelo evoluciono mediante una migracion y no editando la BD a mano.

Migrations/AppDbContextModelSnapshot.cs | Fotografia del modelo actual. | EF Core la compara al crear la siguiente migracion.

Una migracion tiene Up para aplicar y Down para revertir. Se ejecutan en orden usando dotnet ef database update.

6. Proteccion de datos, paso a paso

Secret Manager

La cadena de conexion y clave AES no aparecen en el repositorio. En Program.cs se obtiene ConnectionStrings:DefaultConnection; si no existe, lanza una excepcion con una indicacion clara. User Secrets la proporciona en Development. appsettings.json solo conserva configuracion publica.

Hash de contrasena

La contrasena no se cifra: se transforma en un hash no reversible. Program.cs registra IPasswordHasher<Usuario> con PasswordHasher<Usuario>. En Services/ApiService.cs, metodo CrearUsuario, se ejecuta usuario.PasswordHash = hasher.HashPassword(usuario, request.Password). En actualizacion, UsuariosController.Editar genera otro hash. El metodo privado ADto nunca devuelve password ni PasswordHash. En un futuro login se usaria VerifyHashedPassword.

Cifrado AES-GCM

Telefono y biografia se deben recuperar, por eso se cifran. Services/CifradoService.cs lee Cifrado:Clave, la decodifica desde Base64 y exige 32 bytes (AES-256). Cifrar genera un nonce aleatorio de 12 bytes, codifica el texto a UTF-8 y llama AesGcm.Encrypt. Guarda nonce + tag de 16 bytes + texto cifrado como Base64. Descifrar separa esos mismos segmentos y llama AesGcm.Decrypt; GCM verifica tambien que el contenido no fue alterado.

Uso exacto: PersonajesController.Crear/Editar cifra la biografia; ApiService.Map la descifra. UsuariosController.Crear/Editar cifra telefono; UsuariosController.ADto lo descifra para la respuesta.

Demostracion en MySQL: PasswordHash, Telefono y BiografiaSecreta no contienen el texto original. Demostracion en GET: la API muestra telefono/biografia en claro despues de descifrarlos. Esta diferencia evidencia proteccion en reposo.

Validacion y reglas

- Dtos/Requests.cs exige campos, longitud, email, password minimo de 8 caracteres y poder entre 0 y 100000.
- UsuariosController evita email repetido: 409 Conflict.
- CartasController exige personaje existente: 400 si falta.
- FaccionesController.Personajes controla pagina, tamano y estado.
- PersonajesController.Eliminar protege integridad cuando hay cartas.

7. API completa: rutas, parametros y codigos HTTP

El prefijo comun es /api/v1. Las rutas se escriben en atributos [Route] dentro de Controllers/. Los cuerpos de POST y PUT se envian en JSON.

Personajes - Controllers/PersonajesController.cs

Metodo y ruta | Entrada o comportamiento | Respuesta

GET /personajes | Lista por Id y aplica service.Map para descifrar biografias. | 200, { data: [...] }.

GET /personajes/{id} | Busca por id entero. | 200 o 404.

POST /personajes | { nombre, faccion, estaMuerto, biografiaSecreta }. | 201 Created; 400 si el DTO es invalido.

PUT /personajes/{id} | Mismo cuerpo de POST; reemplaza datos y vuelve a cifrar biografia. | 200, 404 o 400.

DELETE /personajes/{id} | Elimina solo cuando no hay cartas asociadas. | 204, 404 o 400 con mensaje.

Cartas - Controllers/CartasController.cs

Ruta | Comportamiento | Estados

GET /cartas, GET /cartas/{id} | Incluye nombre del personaje con Include(x => x.Personaje). | 200; por id, 404 si falta.

POST /cartas | { personajeId, nombre, poder }, valida personaje existente. | 201, 400.

PUT /cartas/{id} | Edita carta o reasigna a personaje valido. | 200, 404, 400.

DELETE /cartas/{id} | Elimina carta. | 204 o 404.

Eventos - Controllers/EventosController.cs

Ruta | Comportamiento | Estados

GET /eventos, GET /eventos/{id} | Lista por fecha descendente o busca por id. | 200 / 404.

POST /eventos | { nombre, fecha, faccionGanadora }. | 201 / 400.

PUT /eventos/{id}, DELETE /eventos/{id} | Edita o borra por id. | 200, 204 o 404.

Usuarios - Controllers/UsuariosController.cs

Ruta | Proteccion aplicada | Estados

GET /usuarios, GET /usuarios/{id} | Devuelven Id, nombre, email y telefono descifrado; no exponen PasswordHash. | 200 / 404.

POST /usuarios | { nombre, email, password, telefono }; normaliza email, hashea password, cifra telefono. | 201, 400, 409 si email se repite.

PUT /usuarios/{id} | Genera hash nuevo y cifra telefono nuevo. | 200, 404, 400, 409.

DELETE /usuarios/{id} | Elimina usuario. | 204 o 404.

8. Parte propia: Facciones y estadisticas

La logica de estas rutas vive en Controllers/FaccionesController.cs, reutilizando una sola fuente de calculo: Services/ApiService.cs, metodo Estadisticas(). Asi cada endpoint obtiene numeros consistentes.

Ruta | Parametros | Que devuelve o valida

GET /facciones | - | Faccion y cantidad de personajes.

GET /facciones/estadisticas | ordenarPor=poderPromedio, direccion=asc|desc | Todos los FaccionDto y orden opcional por promedio.

GET /facciones/{nombre}/estadisticas | Nombre en ruta. | Una faccion sin distinguir mayusculas; 404 si no existe.

GET /facciones/{nombre}/personajes | estado=todos|vivos|muertos, pagina, tamano. | Lista filtrada y meta; pagina >= 1, tamano 1..50.

GET /facciones/{nombre}/cartas | poderMinimo, orden=asc|desc. | Cartas de los personajes de esa faccion.

GET /facciones/{nombre}/eventos-ganados | desde, hasta opcionales. | Eventos ganados, fecha descendente.

GET /facciones/ranking | criterio=personajes|poderPromedio|eventosGanados, top=1..20. | Posicion, faccion y valor; 400 si criterio/top no son validos.

GET /facciones/comparar | faccionA, faccionB. | Dos DTO y diferencias de personajes/eventos; 400 si nombres iguales, 404 si falta alguna.

GET /estadisticas/generales | - | Totales de facciones, personajes, vivos, muertos, cartas y eventos. Esta en EstadisticasController.cs.

Lectura de ApiService.Estadisticas()

- Carga personajes con cartas usando Include(x => x.Cartas) y carga eventos; ambos AsNoTracking.
- GroupBy(x => x.Faccion.Trim(), StringComparer.OrdinalIgnoreCase) agrupa por faccion, quitando espacios y sin importar mayusculas.
- grupo.Count() da personajes. Los dos Count sobre EstaMuerto dan vivos y muertos.
- Sum(x => x.Cartas.Count) suma cartas.
- SelectMany(x => x.Cartas) aplana listas; si hay cartas, Average(x => x.Poder) entrega el poder medio; si no, devuelve null.
- eventos.Count(...) cuenta ganadores por faccion.
- Math.Round(muertos * 100.0 / total, 2) da porcentaje con dos decimales; 100.0 obliga division decimal.
- OrderBy(x => x.Faccion) deja salida estable y legible.

```
{
"faccion": "Orden Solar",
"personajes": 2, "vivos": 1, "muertos": 1,
"cartas": 2, "poderPromedio": 72.5,
"eventosGanados": 1, "porcentajeMuertos": 50.0
}
```

El contrato se define en Dtos/Responses.cs, record FaccionDto.

9. Pruebas y comprobaciones

Datos de ejemplo y lectura de resultados

La base de prueba tiene personajes de Orden Solar y Alianza Lunar, cartas con poderes 80, 65 y 75, y eventos ganados por ambas facciones. Por eso Orden Solar resulta con 2 personajes, 1 vivo, 1 muerto, 2 cartas, promedio 72.5 y 1 evento ganado. Alianza Lunar resulta con 1 personaje vivo, 1 carta de poder 75 y 1 evento ganado. Los valores vienen de los registros, no estan escritos de forma fija en el controlador.

Prueba | Como repetirla | Resultado esperado

Compilacion | dotnet build o dotnet run. | Compila y arranca. NU1900 puede aparecer si la red no permite consultar auditoria NuGet; no bloquea paquetes ya restaurados.

Migraciones | dotnet ef database update y revisar MySQL. | Se aplican las dos migraciones y se registra el historial.

Propuesta | En Swagger, ejecutar las nueve rutas de facciones/estadisticas. | 200 con datos validos.

Errores controlados | Usar estado=invalido, id inexistente y top fuera de rango. | 400 por entrada invalida y 404 por recurso ausente.

CRUD | Crear, consultar, editar y eliminar un registro temporal en cada recurso. | 201, 200, 200 y 204, segun corresponda.

Hash | Revisar PasswordHash en MySQL despues de crear usuario. | No coincide con el password enviado.

Cifrado | Comparar Telefono/BiografiaSecreta de MySQL con un GET. | MySQL tiene Base64 cifrado; la API responde texto descifrado.

Prueba desde Swagger

- Abre http://localhost:5000/swagger/index.html.
- Abre una ruta y pulsa Try it out.
- Completa parametros o JSON. En usuarios usa email no repetido y password de minimo 8 caracteres.

- Pulsa Execute y revisa Response Code, Response Body y URL.

FacciStats.http contiene ejemplos listos para clientes compatibles: estadisticas, ranking, comparacion, paginacion, un 400, un 404 y resumen general.

Observacion directa en MySQL

```
SELECT Id, Email, PasswordHash, Telefono FROM Usuarios;
SELECT Id, Nombre, BiografiaSecreta FROM Personajes;
```

Estas consultas permiten demostrar almacenamiento protegido. No muestres claves ni contrasenas reales en una presentacion.

10. Guion para defender el trabajo

Presentacion en orden

- Problema: el sistema administra un catalogo y genera indicadores de facciones.
- Arquitectura: controladores reciben HTTP, servicios concentran logica, EF Core accede a MySQL y DTOs limitan entradas/salidas.
- Modelo: cuatro entidades y relacion uno-a-muchos Personaje-Carta.
- Evolucion: dos migraciones: inicial y agregado de biografia cifrada.
- Seguridad: secretos fuera de repo, hash de password, AES-GCM para texto recuperable.
- Aporte: estadisticas, filtros, paginacion, ranking y comparacion.
- Prueba: Swagger para caso correcto, 400/404 y MySQL para contrastar cifrado/hash.

Pregunta probable | Respuesta breve

Por que no guardar password normal? | Una password no debe recuperarse. PasswordHasher guarda un hash con salt y la API no devuelve PasswordHash.

Por que cifrar telefono y no hashearlo? | Debe poder mostrarse de nuevo. AES-GCM permite descifrar con la clave; un hash no.

Que aporta GCM? | Ademas de ocultar, la etiqueta de autenticacion detecta si el dato cifrado fue manipulado.

Donde esta la clave? | En Secret Manager local, nunca en Git ni appsettings.json.

Que hace una migracion? | Versiona el esquema: la primera crea tablas y la segunda agrega una columna; database update las aplica en orden.

Como se relacionan Carta y Personaje? | Por PersonajeId, llave foranea configurada en AppDbContext.OnModelCreating y navegaciones de las entidades.

Que diferencia hay entre 400, 404, 409 y 201? | 400 solicitud invalida; 404 no existe; 409 conflicto como email repetido; 201 creado con URL del recurso.

Demostracion recomendada en cinco minutos

- Muestra el arbol de carpetas y sus capas.
- En Swagger, ejecuta GET /api/v1/facciones/estadisticas.
- Prueba filtro/paginacion: estado=vivos&pagina=1&tamano=10.
- Ejecuta ranking y comparar dos facciones.
- Prueba estado=invalido y un id inexistente.
- Crea un dato temporal sensible, comprueba cifrado en MySQL y lectura descifrada por GET; borralo al terminar.

11. Problemas frecuentes

Situacion | Causa | Solucion

command not found con $env: en Git Bash | Sintaxis de PowerShell usada en Bash. | Usa ASPNETCORE_ENVIRONMENT=Development dotnet run o abre PowerShell.

FacciStats.exe bloqueado | Quedo una API anterior ejecutandose. | Ctrl+C en su consola o finaliza solo ese proceso en Administrador de tareas; vuelve a ejecutar.

Error MySQL | Servicio detenido, puerto incorrecto o secreto ausente/incorrecto. | Inicia MySQL80, verifica 3306 y registra bien la cadena en User Secrets, sin publicarla.

Configura ConnectionStrings... | Falta secreto de conexion. | Ejecuta dotnet user-secrets set en la carpeta del proyecto y reinicia.

Configura Cifrado:Clave... | Falta clave o no mide 32 bytes. | Genera Base64 de 32 bytes. No cambies clave si debes leer datos anteriores.

dotnet ef no reconocido | Herramienta EF no instalada o sin PATH. | Instala/actualiza dotnet-ef para el usuario y repite.

NU1900 | Red no permite acceder a auditoria de NuGet. | No es error de codigo si restore/build finalizan; reintenta con Internet para auditoria.

12. Entrega y control final

- Ejecuta git status: no agregues bin/, obj/, secretos o bases locales.
- Comprueba remoto con git remote -v.
- Comprueba README, USO_DE_IA, migraciones y estructura de capas.
- Antes de exponer, ejecuta build, migraciones y las rutas principales de Swagger.
- Recuerda: claves y passwords son locales y privados; nunca se incluyen en capturas ni entregables.